

Operational Test: The Modify-Right Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone procedure specification, the operational test that verifies whether a given deployment satisfies the **modify right** named as the second of the three rights in the source paper's "human-governed" commitment (§3.1, §3.3, §11.3). A separate derivation note formalizes the modify right as architectural commitment; the present note specifies the procedure by which a deployment is tested for that commitment.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's "human-governed" commitment names three rights — to inspect, to modify, and to override substrate content and orchestration rules at any time during the substrate's existence. An architectural commitment is only as good as the deployment's testability: a system that claims to satisfy the modify right but offers no procedure by which the claim can be checked is not architecturally distinguishable from one that fails it. This note formalizes the **modify-right test** as a standalone operational procedure that verifies whether a deployment satisfies the modify right specifically, outputting a pass/fail result referenceable independently of the inspect-right, override-right, and rule-authoring tests with which it composes. The test detects five canonical violation patterns: LLM-gatekeeping for writes, workflow-approval-gated modifications, scheduled-window restrictions, vendor-managed write restrictions, and read-only-via-vendor-UI patterns. By design, it does not verify inspect, override, rule-authoring quality, provenance completeness, content correctness, or cross-vendor preservation — each of which has a separate procedural test. Comprehensive deployment verification runs all such tests; the present note specifies one of them.

1. Why the modify-right test needs to be formalized as standalone

The modify right — the second of the three rights in the source paper's "human-governed" commitment (§3.3) — is a property of the system's architecture, not a procedural promise made by an operator. Governance is an authority architecture, not a review workflow (§3.3). But an architectural property has

operational consequences only insofar as a deployment can be tested for it. Without a named test procedure, the architectural commitment becomes unobservable; the deployment can claim modify capability without any way to check the claim, and audit, procurement, and composition language has no reliable referent.

A standalone test for the modify right is **separable** (it verifies modify-right satisfaction independently of the inspect, override, and rule-authoring tests, so that a deployment failing one does not entangle the others), **referenceable** (downstream documentation, procurement language, audit checklists, and composition contracts can cite it by name without re-stating its scope), and **deployment-applicable** (it can be performed at four operationally distinct lifecycle points — initial deployment validation, composition partner verification, ongoing audit, and vendor migration verification — yielding the same pass/fail result-shape in each).

This is the second in a series of procedural-test notes formalizing the verification surface of the source paper's architectural commitments.

2. The architectural commitment under test

The modify-right test verifies one architectural commitment: the modify right (§3.1, §3.3). The right has the following operational content under test.

Direct write capability over substrate state. A human with appropriate authorization can modify substrate content (entities, relationships, decisions, rationale, conflicts) and the human-authored orchestration rules that govern cell-level behavior, through a write operation that takes effect as substrate state. The modification is exercised through human-authored orchestration rules: the rule-authoring path the source paper names (§3.3) as the design-time governance moment.

Architectural property, not procedural promise. The right is satisfied as a property of the system's design, not as a vendor's current policy or operator's current practice. A deployment in which a vendor, runtime middleware, or LLM-mediated workflow can in principle prevent or transform a human's modification fails the architectural commitment, regardless of how rarely such prevention occurs in current operation (§3.3). The test must probe what the system permits in principle, not only what occurs at evaluation time.

Temporal property: at any time. The right is exercisable when the human chooses, not when a workflow permits it. Scheduled review windows, approval-gated sessions, and time-locked write portals all fail the temporal component, even when they grant ample modify access otherwise (§3.3).

Authoritative effect: substrate state change. The right is exercised when the modification produces a change to authoritative substrate state, not only to a derived view, vendor cache, or transient projection (§11.3). The substrate is the source of truth; a modification that affects only a non-authoritative surface does not satisfy the right's operational content.

These four define what the modify right requires of the deployment under test. The procedure in §3 operationalizes the verification of each.

3. The test procedure as operational steps

The modify-right test consists of five operational steps, performed against the substrate under test, in the deployment's actual configuration, with appropriate authorization configured for the test subject.

Step 1 — Author or select a rule under which the modification is performed. The evaluator authors (or selects an existing) human-authored orchestration rule that governs the write operation. The rule is itself a substrate-level artifact whose authorship lineage identifies a human. This grounds the modification in the rule-authoring path (§3.3). If the deployment does not permit the evaluator to author or select a rule directly under appropriate authorization, the test fails at Step 1.

Step 2 — Perform a write to substrate state under the rule. The evaluator performs a write — a content edit, addition, deletion, or structural change — through the rule selected in Step 1. The write request must reach the substrate without intermediate transformation by an LLM. An LLM may serve as adjacent tooling (drafting suggestions, validating syntax, formatting); it cannot stand between the evaluator's write intent and the substrate as a gate or transformer. If the only path runs through an LLM that produces a summary, paraphrase, or reformulation of the write before substrate is updated, the test fails at Step 2.

Step 3 — Verify the substrate state change through subsequent inspection. The evaluator reads substrate state directly (using the inspect right tested separately) to confirm the write took effect — the new state reflects what was written, in inspectable form, in the substrate proper rather than only in a derived view. If the modification appears in a vendor surface, cache, or projection while the underlying substrate is unchanged, the test fails at Step 3 (§11.3).

Step 4 — Confirm the write was available at the time of the evaluator's choosing. The evaluator records whether the write proceeded immediately or required scheduling, an approval-gated review, or a workflow step. If the modification could be executed only during a designated window, only after a documented approval routed through additional reviewers, or only at a vendor's scheduled cadence, the test fails at Step 4. A write that succeeds only after a delay imposed by approval workflow does not satisfy the at-any-time component.

Step 5 — Confirm provenance records the human author. The evaluator inspects the substrate's provenance metadata and verifies the change is attributed to the authorized human who performed Steps 1–2. The modify-right test does not verify provenance completeness in full — that has a separate test — but it does verify that the modification is attributed at all, since an unattributed write fails the substrate-as-source-of-truth commitment in a way the modify right cannot be exercised meaningfully without.

A deployment passes the test if and only if all five steps complete successfully under the deployment's actual configuration.

4. What the test outputs

Pass. All five steps complete successfully. The deployment is verified to satisfy the modify right at the time and scope of the test. The pass is local: it does not certify the inspect, override, or rule-authoring

commitments separately, nor modify-right satisfaction outside the tested scope.

Fail. One or more steps do not complete successfully. The failure attribution names the step (Step 1: rule authoring not permitted; Step 2: LLM-mediated or otherwise transformed write; Step 3: write affects only a derived view; Step 4: scheduled or approval-gated; Step 5: provenance does not attribute the human author) so downstream remediation can be scoped. Multiple failures are possible in a single run; each is reported.

Inconclusive. The test reports inconclusive only when an operational obstacle prevents completion of a step that is not itself the property under test (the substrate is unreachable due to an unrelated outage; an authorization configuration change is in flight). Inconclusive must be re-run before pass/fail is recorded.

The pass/fail vocabulary is intentionally narrow. The architectural commitment is binary by design: a system that fails any of the four components in §2 does not satisfy the modify right specifically, even if it satisfies the broader human-governed commitment in some other respect (§3.3).

5. What anti-patterns the test detects

The test detects five canonical anti-patterns that violate the modify right specifically. Each has independent treatment elsewhere in the present derivation program.

LLM-gatekeeping for writes. The deployment routes the human's write intent through an LLM that summarizes, paraphrases, or transforms the request before substrate is updated. The LLM is the gate; the human's authority is mediated through the model's output rather than expressed directly to the substrate. Step 2 fails. The pattern is common in deployments that wrap substrate writes inside conversational agents; the architectural cost is that the LLM becomes a structural authority over what is committed.

Workflow-approval-gated modifications. The deployment requires the modification to traverse an approval workflow — a designated reviewer queue, a multi-step ticketing pipeline, a release-management gate — before substrate is updated. Step 4 fails. This is the canonical modify-right violation because it presents itself as governance (review!) while in fact replacing architectural authority with procedural gating; governance properties that depend on workflow rather than architecture are not governed in the source paper's sense.

Scheduled-review-window restrictions. The deployment permits modifications only during designated time windows — quarterly reviews, weekly change-management slots, scheduled maintenance windows. Step 4 fails. The modify right becomes a property of the calendar rather than of the system.

Vendor-managed write restrictions. The deployment routes writes through a vendor system that can in principle prevent or transform modifications without explicit reversibility by the operator. The architectural commitment is to a property of the system's design, not to a vendor's current policy; the test fails at Step 2 (vendor transforms the write) or Step 3 (vendor accepts but does not propagate to authoritative substrate).

Read-only-via-vendor-UI patterns. The deployment exposes substrate content through a vendor user interface that permits reading but not writing, even when the substrate is conceptually modifiable. The human must request changes through a separate channel — a support ticket, a configuration change request, a vendor administrator. Step 2 fails because the human's authorized direct write is architecturally prevented at the access surface, regardless of whether some other party eventually commits a corresponding change.

A deployment that exhibits none of these and passes Steps 1–5 satisfies the modify right at the test scope.

6. How the test integrates with deployment verification

The procedure in §3 is the same at four operationally distinct lifecycle points; the integration differs in trigger and scope.

Initial deployment validation. Before activation, the test is run as part of architectural-commitment verification, against representative substrate content, rules, and authorizations. A failure here is a blocker for activation; downstream commitments that depend on the modify right (override, rule authoring, conflict handling) are likewise unsatisfied until remediation.

Composition partner verification. When a CKS substrate is composed with another substrate, an adjacent system, or a partner deployment, the test is re-run against the composed configuration. Composition can introduce mediated-write patterns the host deployment passed in isolation: a partner's API requires approval routing; a partner's vendor introduces a write transformation; a partner's substrate is read-only at the composition surface. This integration is what makes governance preservation across composition operationally checkable rather than merely asserted.

Ongoing audit. The deployment's governance function runs the test periodically, at a cadence set by organizational policy. Periodic re-running detects regressions introduced by configuration drift, vendor policy change, runtime middleware update, or organizational role reassignment. The audit form typically samples scopes rather than testing exhaustively.

Vendor migration verification. When a vendor underlying the deployment is changed — a new LLM provider, substrate host, or orchestration runtime — the test is re-run after migration. Migrations are a frequent source of regressions: a new vendor may introduce a write-transformation step, different approval routing, or a more restrictive access model. This integration is what makes vendor-substitutability claims verifiable rather than nominal.

The four points are not redundant — each detects a class of regression the others do not.

7. Limits of the test

Stating precisely what the modify-right test does not verify is what keeps it scoped to the modify right specifically.

Not the inspect right. A deployment that passes the modify-right test may still fail the inspect right. Modify and inspect are separable; a system that permits writes but gates reads is not human-governed in the joint sense, but the failure is not detected by the modify-right test.

Not the override right. A deployment that passes the modify-right test may still fail the override right's no-justification-required component. The override right has independent operational content the modify-right test does not probe.

Not rule-authoring quality. Step 1 requires a human-authored rule but does not verify the rule was authored under the conditions the source paper specifies (not LLM-committed in a manner that bypasses human authority, versioning intact, authorship attributable across revisions). A deployment whose rule-authoring is compromised but whose rule-mediated write path operates correctly will pass this test and fail the rule-authoring test; both are required for joint human-governed satisfaction.

Not provenance completeness. Step 5 verifies attribution exists; it does not verify the full provenance specification (lineage chain integrity, retraceability across rule revisions). Provenance completeness has a separate test.

Not rule logic correctness. A rule may be authored, write to substrate, take effect, and produce attributed provenance while encoding an operationally incorrect processing decision. Rule correctness is not a modify-right concern; the right is access, not correctness.

Not cross-vendor modify preservation. A deployment that passes against one vendor configuration may still fail to preserve modify capability across migrations. The migration-integration form (§6) probes a single transition; cross-vendor preservation as a sustained property has its own procedural test.

The limits are the boundary that makes the test referenceable. Comprehensive verification runs all procedural tests in the series in sequence.

8. One-sentence test

A deployment satisfies the CKS modify right if and only if an authorized human, operating under a human-authored orchestration rule, can perform a write that produces a verifiable change to authoritative substrate state, attributed to the human author, at the human's chosen time, without LLM intermediation, without workflow-approval gating, without scheduled-window restriction, and without vendor-managed write transformation or read-only-via-vendor-UI obstruction.

9. Conclusion

Treating the modify-right test as a standalone procedure specification produces three downstream affordances. Procurement language, vendor contracts, and composition agreements can cite a precisely-scoped test rather than a composite "supports human governance" phrase that resists enforcement. Failure attribution becomes precise: a deployment failing the human-governed commitment can be diagnosed at the specific test that failed. And lifecycle-integrated testing becomes coherent: the four integration points produce results in the same shape regardless of when in the lifecycle the test is run.

This is the second of approximately sixteen procedural-test notes in the present derivation series. The first formalized the inspect-right test; the next will formalize the override-right test. Subsequent notes will cover rule-authoring, the AI-as-substrate-mediator role, the determinism contract, retraceability and provenance, source-of-truth, tool-agnosticism, cross-vendor preservation, the cost contract, conflict-as-first-class handling, composition requirements, pattern-mapping, and reproducibility. Together, the procedural tests specify the verification surface of the source paper's architectural commitments — the boundary at which the architecture's claims become operationally checkable rather than only declaratively asserted.

Subsequent work that implements, extends, audits, or argues against the CKS modify right should use the modify-right test in the sense formalized here. Work that uses the term differently is testing a different property, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Operational Test: The Modify-Right Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.